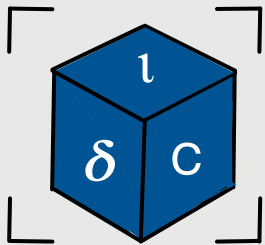


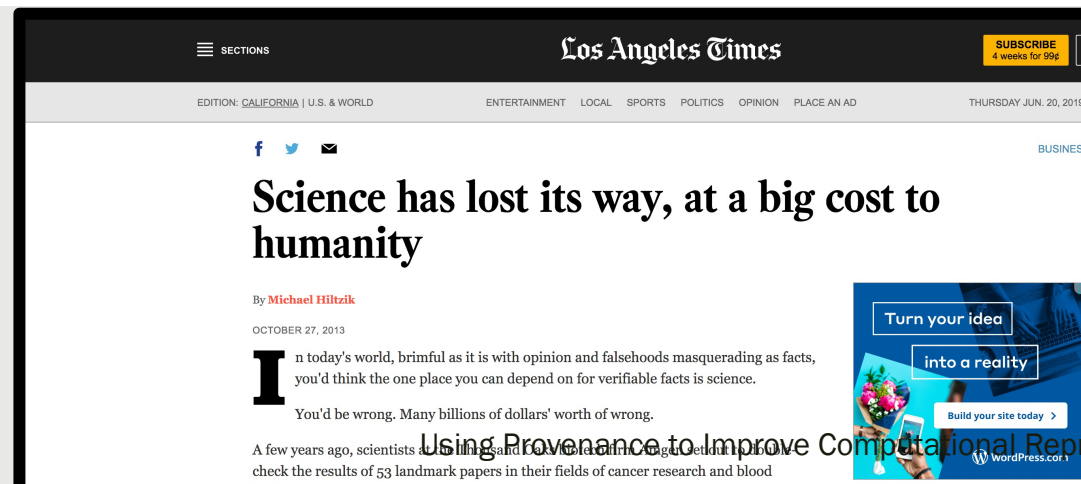
USING PROVENANCE TO IMPROVE COMPUTATIONAL REPRODUCIBILITY

Tanu Malik

The DICE Laboratory
School of Computing
DePaul University, Chicago IL



Reproducibility and Replicability in Sciences: A growing concern





The Harvard Professor and the Bloggers

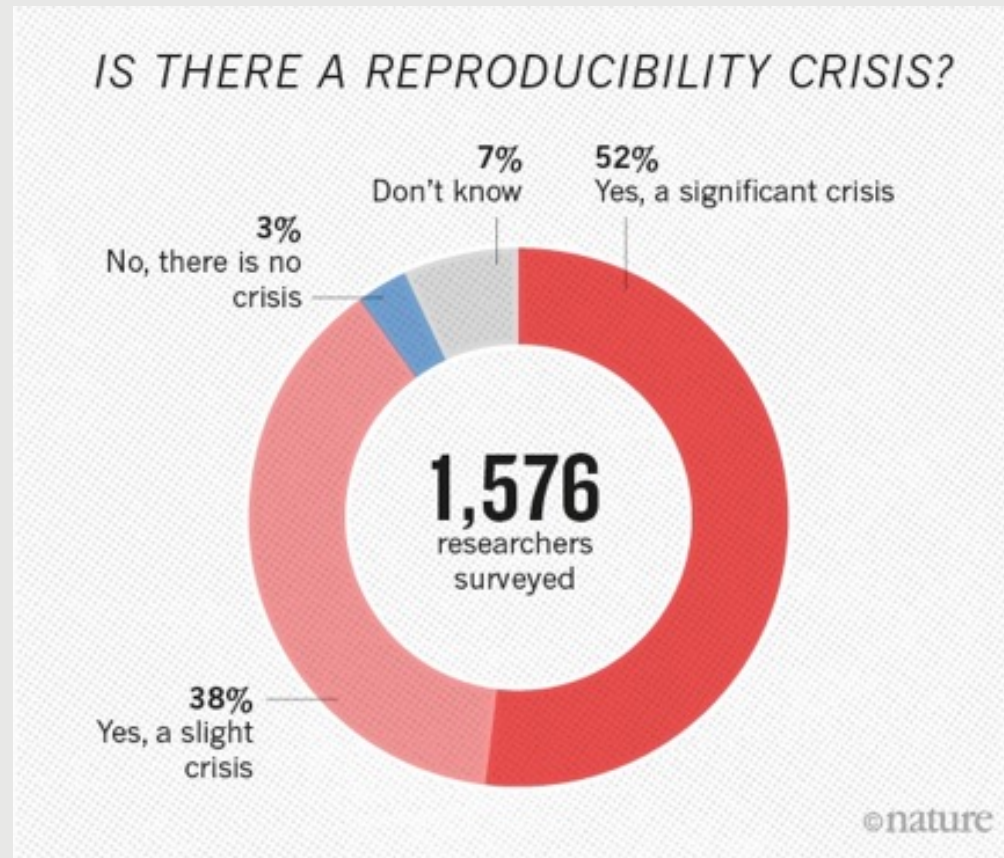
When Francesca Gino, a rising academic star, was accused of falsifying data — about how to stop dishonesty — it didn't just torch her career. It inflamed a crisis in behavioral science.

Stanford President Will Resign After Report Found Flaws in His Research

Marc Tessier-Lavigne was cleared of accusations of scientific fraud and misconduct. But the review said his work had “multiple problems” and “fell below customary standards of scientific rigor.”



A Survey to Validate Concerns



Source: 1500 scientists lift the lid on reproducibility. *Nature Survey*. Accessed 25th May, 2016





Concerns in Computer Sciences

- Definition: Given an experiment described at t , Can it be downloaded at t' , and its source code be built within a reasonable amount of time.

N = 402 experiments

Response	Percentage	# of experiments
No response to code requests	36.1%	145
Declined to provide code	10.2%	41

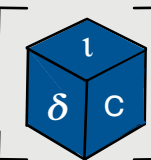
N = 219 experiments

Response	Percentage	# of experiments
Failed to build	5.02%	11
Built $\leq$ 30 minutes	32.4%	71
Built $>$ 30 minutes	15.9%	35
Reasonable effort	46.6%	102

T. Proebsting, A. M. Warren, and C. Collberg. 2015.

Repeatability and benefaction in computer systems research. University of Arizona TR 14.Vol. 4. 1–68.

Using Provenance to Improve Computational Reproducibility, ReWords, eScience, 2023





Challenges in Reproducibility and Replicability

- **Reproducibility:** Obtaining consistent results when using the same or similar input data, computational steps, conditions of analysis, etc.
- **Replicability:** Obtaining consistent results when using different input data, computational steps, conditions of analysis, etc.

NSF 23-018

Dear Colleague Letter: Reproducibility and Replicability in Science

October 25, 2022

National Academies of Sciences, Engineering, and Medicine 2019. Reproducibility and Replicability in Science. Washington, DC: The National Academies Press. <https://doi.org/10.17226/25303>.





Challenges in Reproducibility and Replicability

- **Reproducibility:** Obtaining consistent results when using the same or similar input data, computational steps, conditions of analysis, etc.
- **Replicability:** Obtaining consistent results when using different input data, computational steps, conditions of analysis, etc.

Challenges

- Need for **guarantees-based or statistical-based methods** for conduct of reproducible research
- Need for **infrastructure** that supports reproducible research
- Need for **policies** that incentivize and enforce reproducible research

National Academies of Sciences, Engineering, and Medicine 2019. Reproducibility and Replicability in Science. Washington, DC: The National Academies Press. <https://doi.org/10.17226/25303>.





Foundations of Reproducible Scientific Computing

Guarantees

Lineage-based methods

Replay: VLDB'22,

Debugging: CIKM'23,
HiPC'22, TaPP'20

Guarantees: TaPP'13,
ICDE'15, MDPI'18

Infrastructure

Container-based Tools

w/ Lineage: eScience'22
Best Student Paper,
eScience '19, eScience'17

Size Reduction: Access'23,
PARCO'20, HiPC'20,

Vs Docker: ICCS'15

Policies

Artifact Evaluation

Cloud Experimentation:
CloudSummit' 23

Surveys: PRECS'22, IEEE
CiSE'21

<https://dice.cs.depaul.edu>





Foundations of Reproducible Scientific Computing

Guarantees

Lineage-based methods

Replay: VLDB'22,

Debugging: CIKM'23, HiPC'22, TaPP'20

Guarantees: TaPP'13, ICDE'15, MDPI'18

Infrastructure

Container-based Tools

w/ Lineage: eScience'22
Best Student Paper,
eScience '19, eScience'17

Size Reduction: Access'23, PARCO'20, HiPC'20,

Vs Docker: ICCS'15

Policies

Artifact Evaluation

Cloud Experimentation:
CloudSummit' 23

Surveys: PRECS'22, IEEE CiSE'21





Share and Reproduce an Application



Alice



Bob

Alice wants to share her input data files and program source code with Bob
Bob wants to reproduce Alice's application to validate her outputs.





Repeat Share and ~~Reproduce~~ an Application



Alice wants to share her input data files and program source code with Bob
Bob wants to repeat Alice's application to validate her inputs and outputs.





Alice's sharing options

1. Email a tar/gzip
2. Build a website with model code, parameters, and data
3. Create a virtual machine or container





Not sufficient for reproducible research

- ~~1. Email a tar/gzip~~
- ~~2. Build a website with model code, parameters, and data~~
- ~~3. Create a virtual machine or container~~





Not sufficient for reproducible research

- ~~1. Email a tar/gzip~~
- ~~2. Build a website with model code, parameters, and data~~

Missing environment files: No isolation guarantee

- ~~3. Create a virtual machine or container~~

Failing rebuilds: No repeatable guarantee





Docker for repeating an application?

```
# syntax=docker/dockerfile:1
FROM golang:1.16-alpine AS build

# Install tools required for project
# Run `docker build --no-cache .` to update dependencies
RUN apk add --no-cache git
RUN go get github.com/golang/dep/cmd/dep

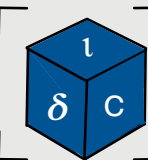
# List project dependencies with Gopkg.toml and Gopkg.lock
# These layers are only re-built when Gopkg files are updated
COPY Gopkg.lock Gopkg.toml /go/src/project/
WORKDIR /go/src/project/
# Install library dependencies
RUN dep ensure -vendor-only

# Copy the entire project and build it
COPY . /go/src/project/
RUN go build -o /bin/project

# This results in a single layer image
FROM scratch
COPY --from=build /bin/project /bin/project
ENTRYPOINT ["/bin/project"]
CMD ["--help"]
```

Because this line installs the most recent version upon rebuilding

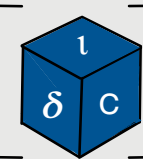
This line no longer rebuilds





Can we compose Containerization and Lineage?

<http://sciunit.run>





Sciunit: Compose Containerization and Lineage

<http://sciunit.run>

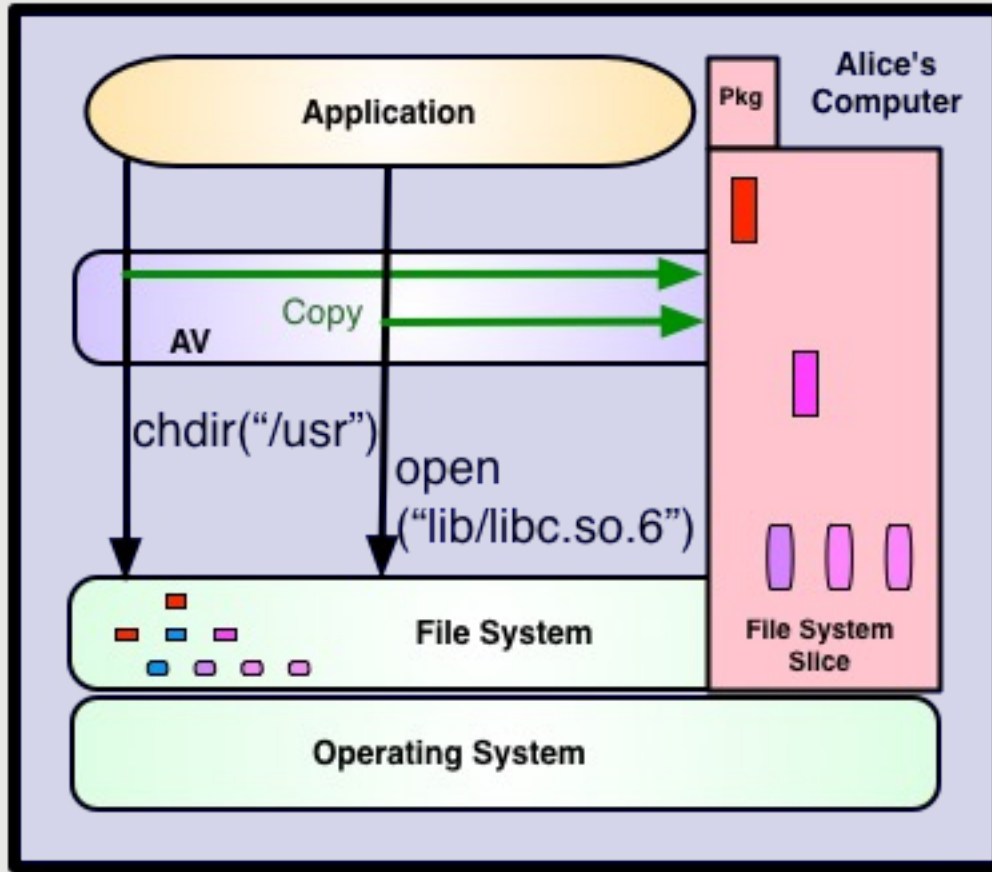
Key Idea: During program execution identify and isolate data dependencies and infer lineage between dependencies

D.H. Ton That, G. Fils, Z. Yuan, T. Malik. Sciunits: Reusable Research Objects.
In IEEE eScience Conference (eScience), 374-383, 2017





Host: Use *ptrace* to observe executions



- Audit ~50 system calls related to process control, file I/O, and network
 - If file is `/dev/random` capture return bytes as well
- At the time of interception:
 - Generate an execution trace of system call events in real-time
 - Copy files mentioned as part of system calls into a container

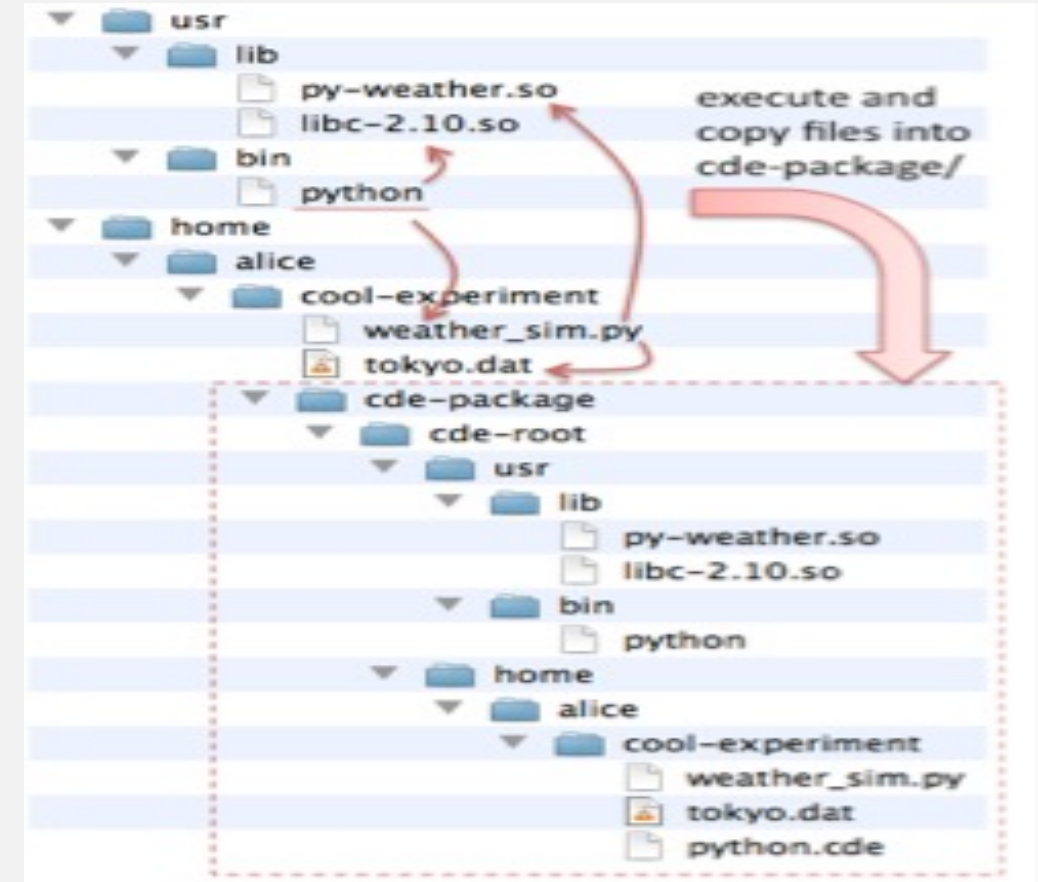
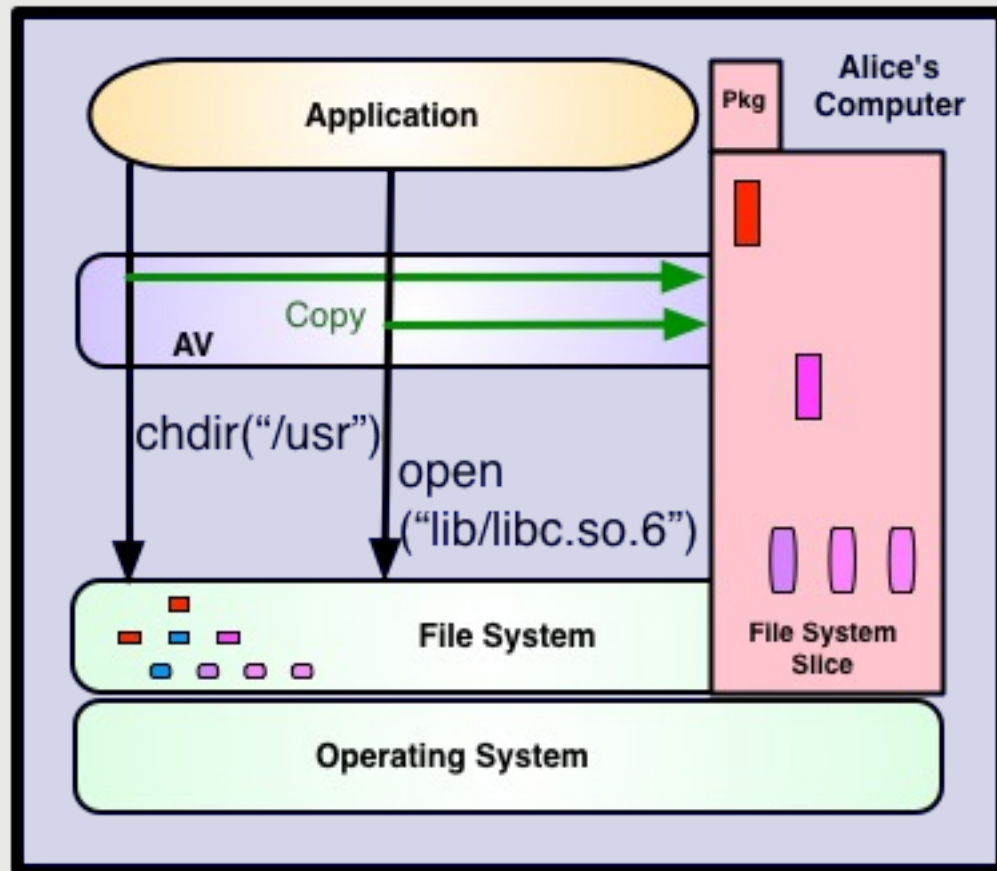
Q.Pham, T. Malik, I.Foster, PTU: Provenance to Use, In USENIX Theory and Practice of Provenance, 2013





Create a *chroot*-based container

- Audit provenance during container creation time





Alices shares *sciunits* and Bob repeats them



Alice



sciunits



Bob

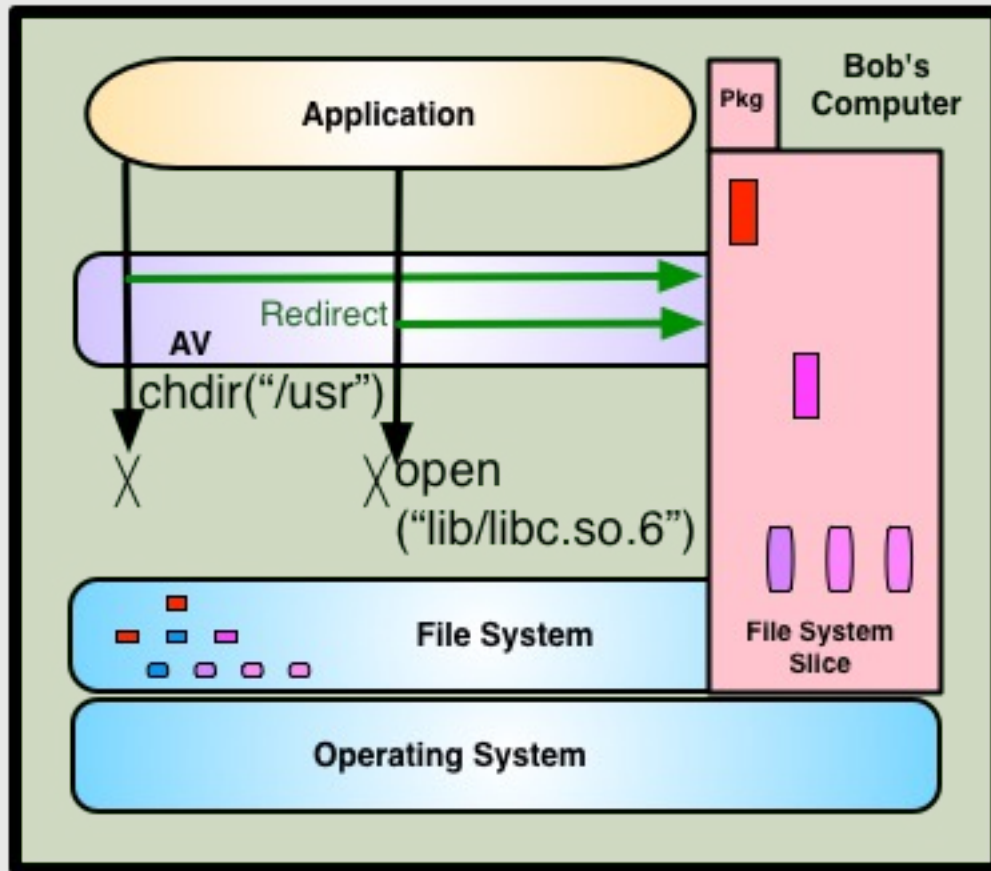




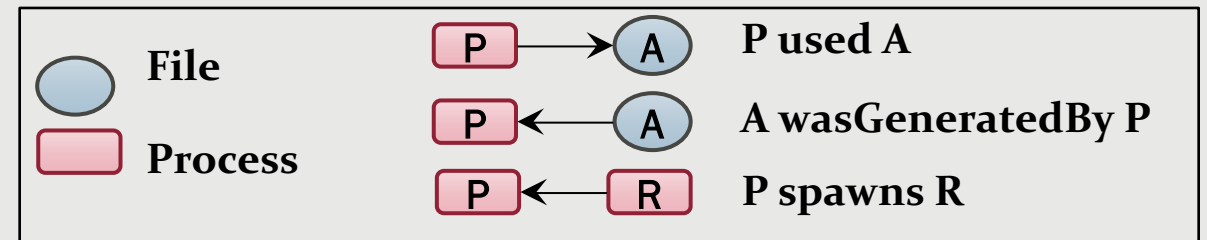
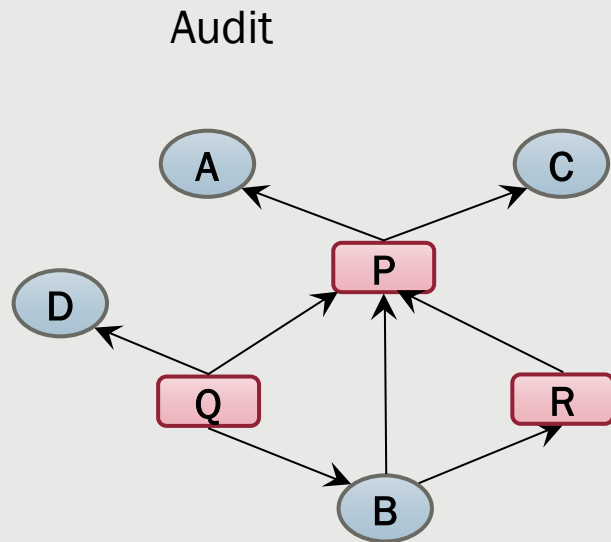
Target: Use *ptrace* to redirect executions into the container

Repeat Phase

- Redirection during repetition is only for file- and network-related system events.
- Repeat execution happens within a process and file namespace.

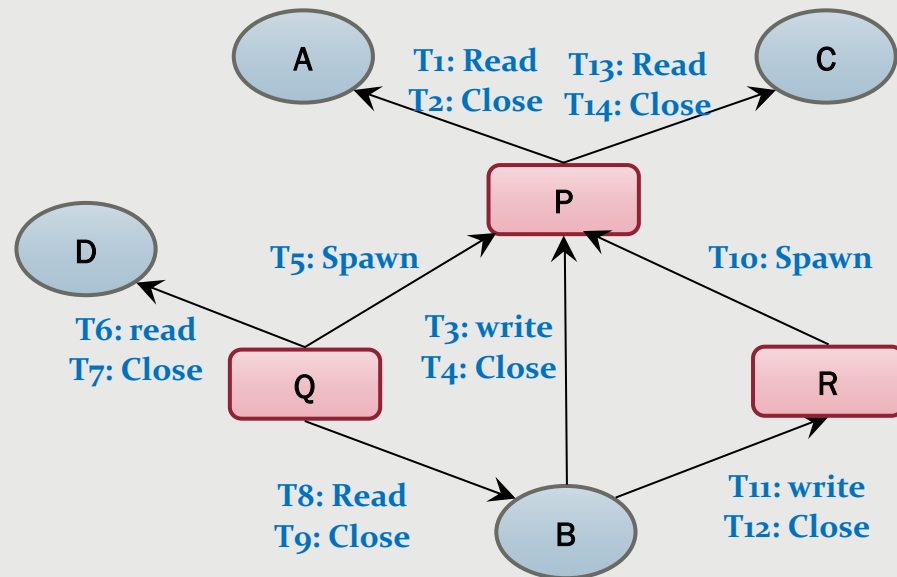


Mapping system events to a provenance graph

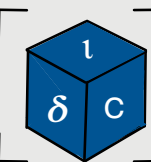
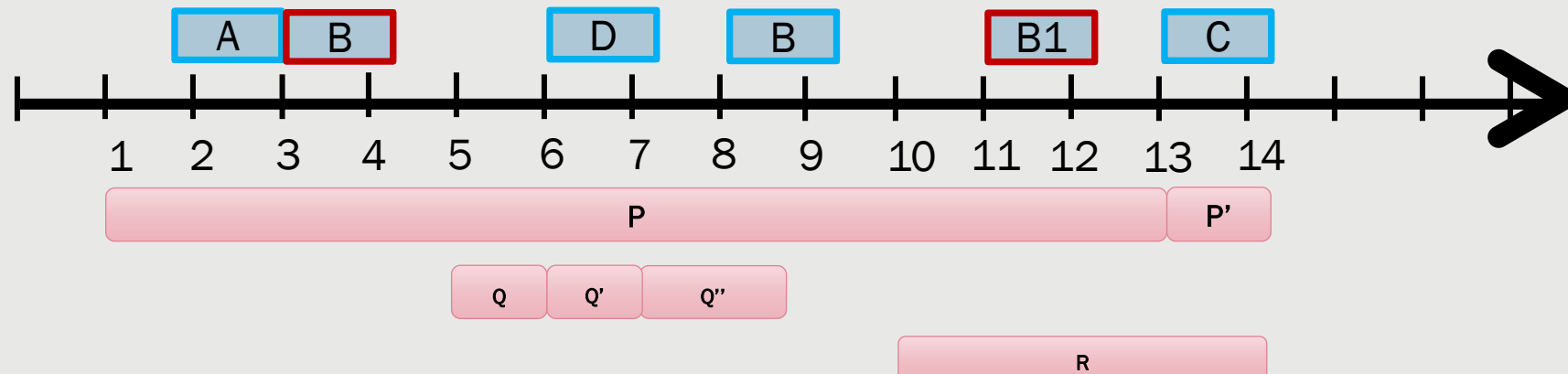




Versioning of process and file nodes



- Each file is versioned if it is written over
 - Write on a file changes file content
- Each process if versioned if it reads a new file
 - Read in a process changes process state





Execution Trace

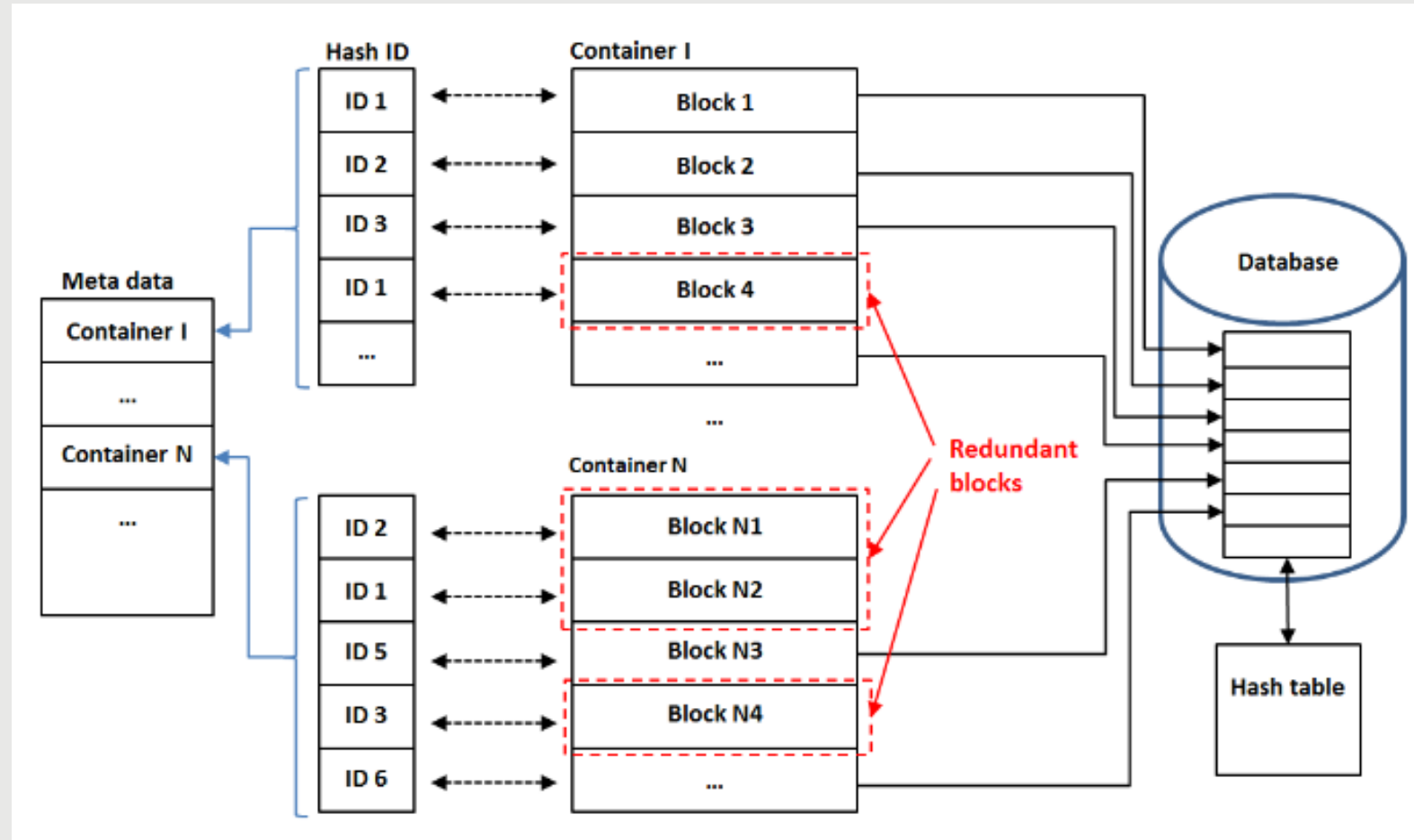
- An execution trace T for program P is a 2-tuple $\langle G, Z \rangle$, where G is the provenance graph and Z is the package
- T is deterministic, i.e., it will lead to the same result, $R = \{R_1, \dots, R_n\}$, every time $t' > t$ iff
 - *$T(G)$ includes necessary activities, entities and edges, i.e., it does not leave out activities, entities and edges in $G(V, E, T)$ that may have caused Z*
 - *$T(Z)$ is sufficient, i.e., it does not include superficial elements that do not cause R*

A. Youngdahl, D.H. Ton That, T. Malik, "Scilnc: A Container Runtime for Incremental Recomputation", In *IEEE 15th International Conference on eScience (eScience)*, 291-300, 2019





Container Storage



- Container files are deduplicated using Rabin hash

Evaluation



- Settings:
 - *Workstation: Intel core i7-3770 3.4Ghz (8 cores), 20GB RAM, 1TB SATA HDD, running Ubuntu 18.04 64 bit.*
 - *Use cases:*
 - FIE: Chicago Food Inspections Evaluation (~ 307 MB)
 - VIC: Variable Infiltration Capacity (~ 1.2 GB)
 - IQE: Incremental Query Execution (~ 22 MB)
 - *Base Lines:*
 - Docker
 - IncPy





Use case Description

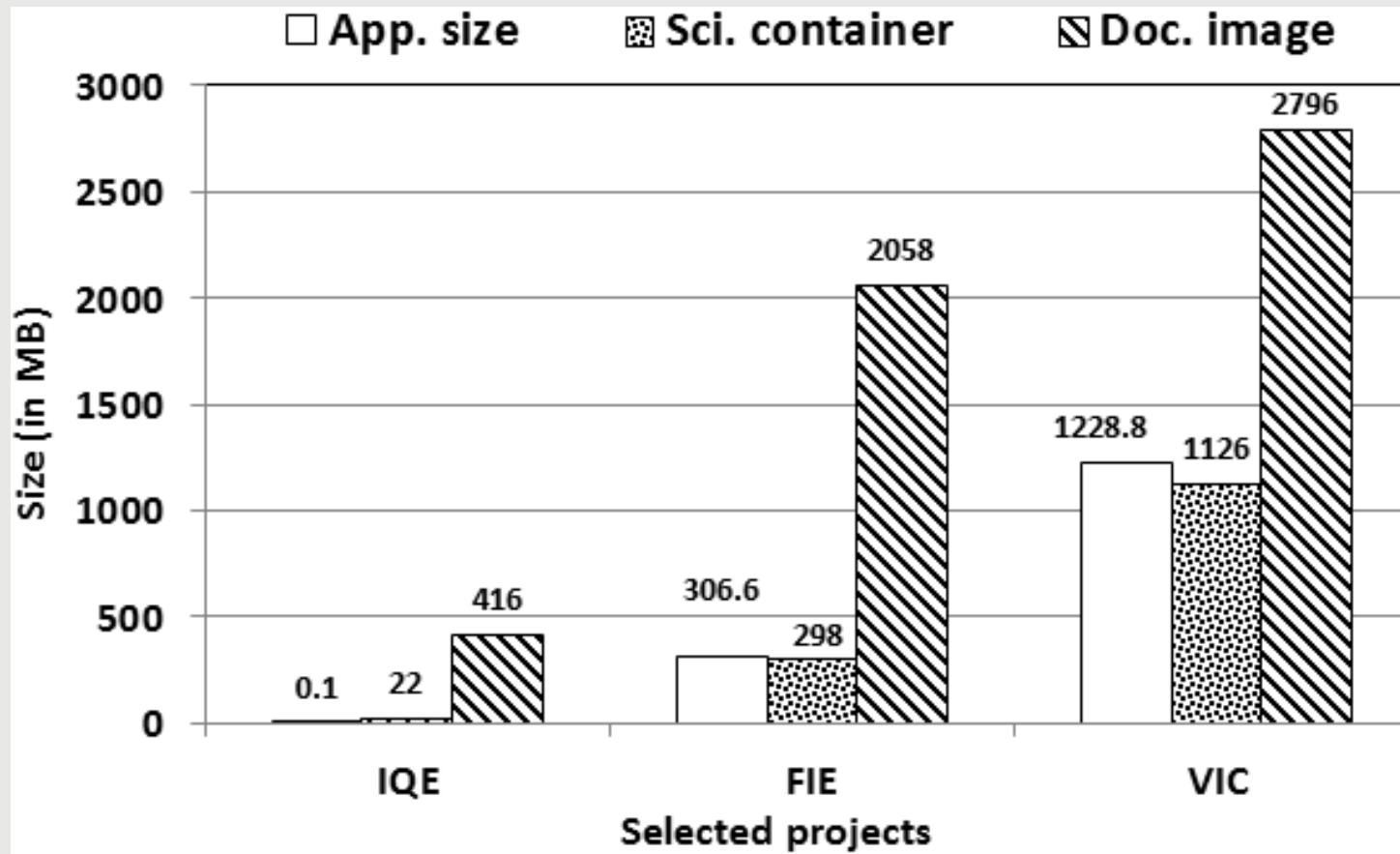
TABLE I: Usecases descriptions.

	FIE [16]	VIC [17]	IQE [18]
Source code languages	R, Bash	C, C++, Python, C shell script, Fortran	Python
Source code files	29	97	5
Data files	14	11,481	5
Dependency files	659	357	112
Size of all files	306.6 MB	1.2 GB	22 MB
Normal run time	286.756 s	40.259 s	5.226 s

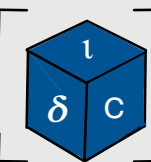




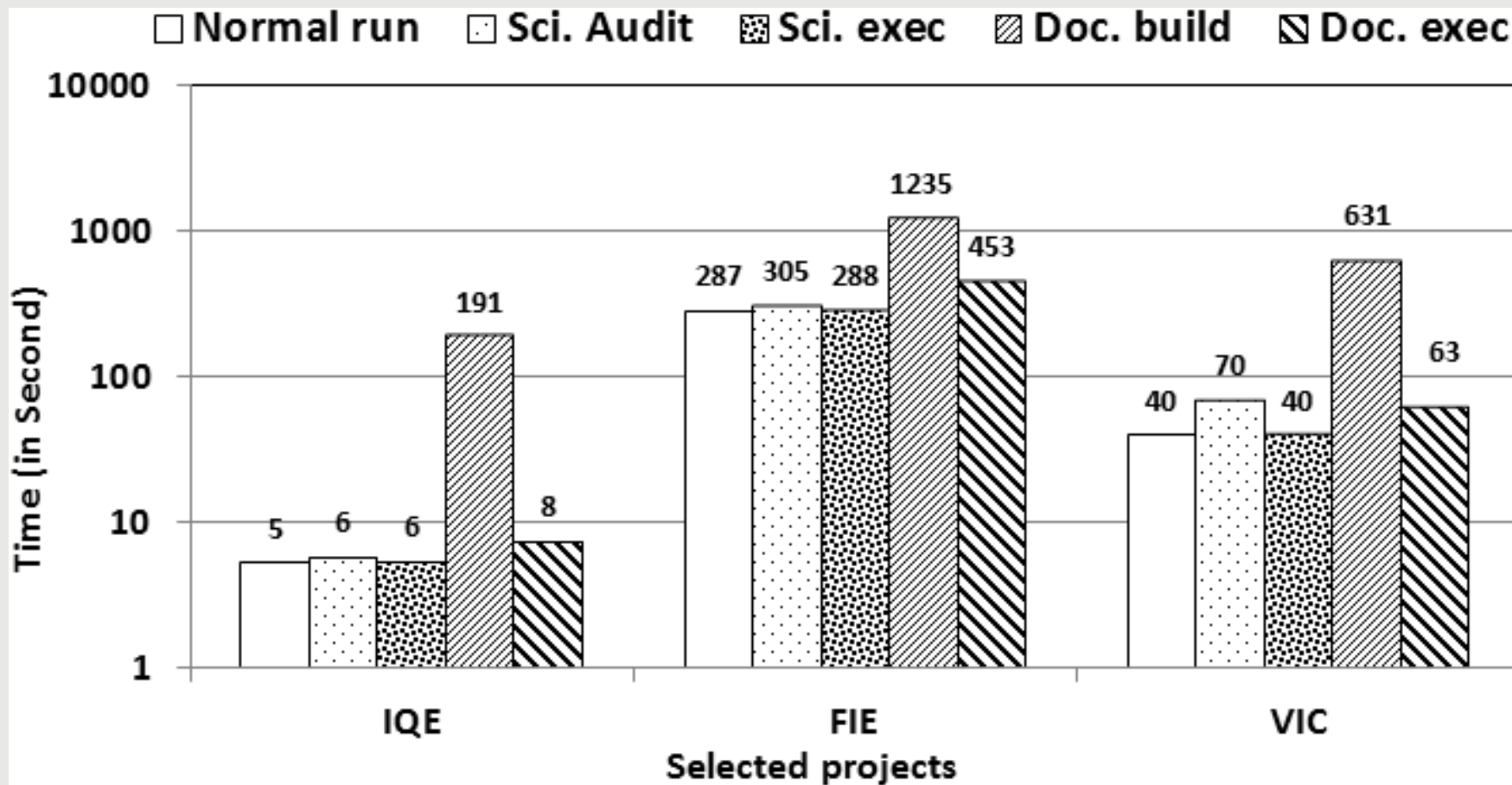
Container size



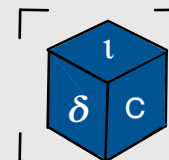
1. Docker containers are 19X, 7X and 2.5X larger than those of Sciunit.
2. Sciunit containers are even slightly smaller than the original application package size.



Auditing and re-execution time



1. Docker spends 4X and 9X longer time than Sciunit to build FIE and VIC container, 7X and 2.5X larger.
2. Sciunit only spends slightly higher time than the original execution time to build containers.





Sample Interaction of Sciunit

```
1. > sciunit create FIE
2. > sciunit exec ./FIE.sh ./DATA/weather_201710.Rds
   0. Download...
   1. Calculate violation matrix...
   2. Calculate heat map...
   3. Generate model data with ./DATA/weather_201710.Rds...
   4. Apply random forest model...
   5. Evaluation...
3. > sciunit list
   e1 Dec 4 12:44 ./FIE.sh ./DATA/weather_201710.Rds
4. > sciunit show
   id: e1
   sciunit: FIE
   command: ./FIE.sh ./DATA/weather_201710.Rds
   size: 306.6 MB
   started: 2017-12-04 12:44
5. > sciunit push
   ...
   Title for the new article: FIE
   new: 306.6 MB [01:05, 4.72MB/s]
6. > sciunit copy
   mSLLTj#
```

Alice's Computer

```
1. > sciunit open mSLLTj#
   Opened Sciunit FIE
2. > sciunit list
   e1 Dec 4 12:44 ./FIE.sh ./DATA/weather_201710.Rds
3. > sciunit repeat e1
   ...
   0. Download...
   1. Calculate violation matrix...
   2. Calculate heat map...
   3. Generate model data with ./DATA/weather_201710.Rds
   4. Apply random forest model...
   5. Evaluation...
4. > sciunit given '/tmp/weather_201801.Rds' e1 %
   ...
   1. Generate model data with '/tmp/weather_201801.Rds'
   2. Apply random forest model...
   3. Evaluation...
5. > sciunit list
   e1 Dec 4 12:44 ./FIE.sh ./DATA/weather_201710.Rds
   e2 Dec 14 2:44 ./FIE.sh ./tmp/weather_201801.Rds
```

Bob's Computer



Notebook is Interactive and Shareable



```
[ ]: plt.subplots(figsize =(12, 8))
x = np.arange(len(df2.index))

[16]: plt.bar(x, df2['Size 2'], width=bar_width, hatch='-')
plt.bar(x+(1*bar_width), df2['Size 1'], width=bar_width, hatch='x')
plt.bar(x+(2*bar_width), df2['Size 3'], width=bar_width, hatch='/')

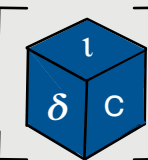
[16]: <BarContainer object of 5 artists>
```

Category	Size 1 (MBs)	Size 2 (MBs)	Size 3 (MBs)
0	~100	~400	~900
1	~100	~400	~900
2	~100	~400	~900
3	~100	~1800	~3000
4	~100	~600	~1200

```
[ ]: plt.ylabel('Container Size (MBs)', fontsize=18, fontweight='bold')
plt.xticks([r + bar_width for r in range(len(x))], df2.index)
plt.legend(labels=labels)

[18]: plt.savefig('export_size_containers_all.png', dpi=500)
df2['Size 2'].head(3)

[18]: Use Case
FlowFromSnow    72.7
HAND             72.7
NatGas           72.7
Name: Size 2, dtype: float64
<Figure size 432x288 with 0 Axes>
```





Notebook is like a “Container”

Shared

Code Cells

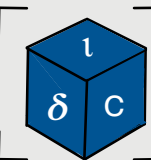
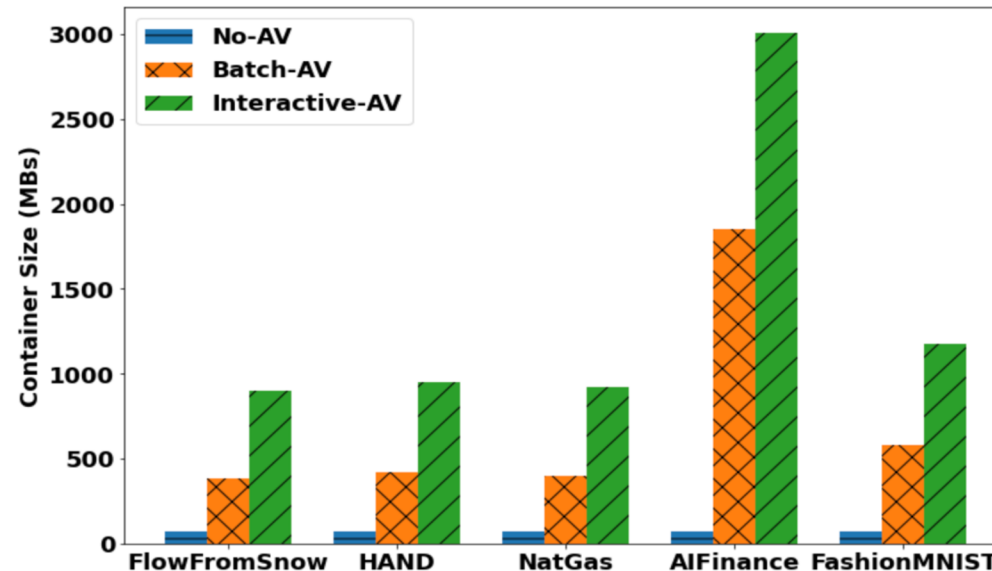
Data Results

Visualizations

```
[9]: plt.subplots(figsize =(12, 8))
x = np.arange(len(df2.index))
plt.bar(x, df2['Size 2'], width=bar_width, hatch='-')
plt.bar(x+(1*bar_width), df2['Size 1'], width=bar_width, hatch='x')
plt.bar(x+(2*bar_width), df2['Size 3'], width=bar_width, hatch='/')
```

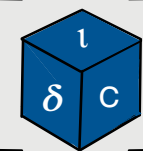
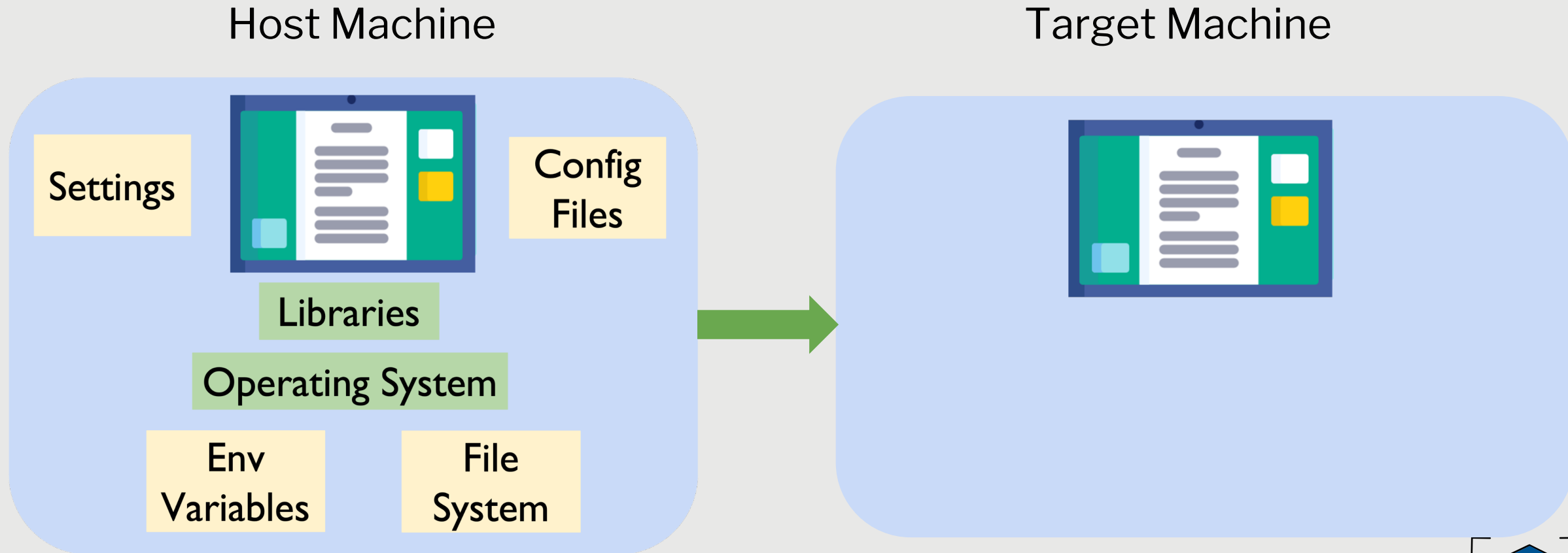
```
[9]: plt.ylabel('Container Size (MBs)', fontsize=18, fontweight='bold')
plt.xticks([r + bar_width for r in range(len(x))], df2.index)
plt.legend(labels=labels)
plt.savefig('export_size_containers_all.png', dpi=500)
df2['Size 2'].head(3)
```

```
[9]: Use Case
FlowFromSnow    72.7
HAND             72.7
NatGas           72.7
Name: Size 2, dtype: float64
```





Notebook Environment is Not Shared

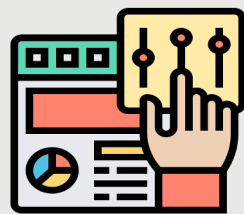


FLINC

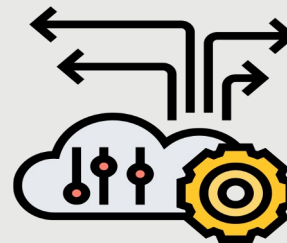
R. Ahmad, N. Manne, T. Malik. Reproducible Notebook Containers with Application Virtualization,
In *IEEE eScience*, Vol 5(1), 2022. Best Student Paper Award



- Creates lightweight notebook containers in an automated manner which are easy to use and share.
 - Preserves the interactive nature of notebooks
 - Guarantees reproducible execution
 - Supports flexibility to update and modify notebooks.



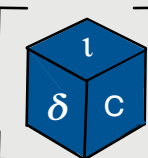
Preserve
Interactivity



Flexible for
Modifications

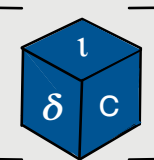
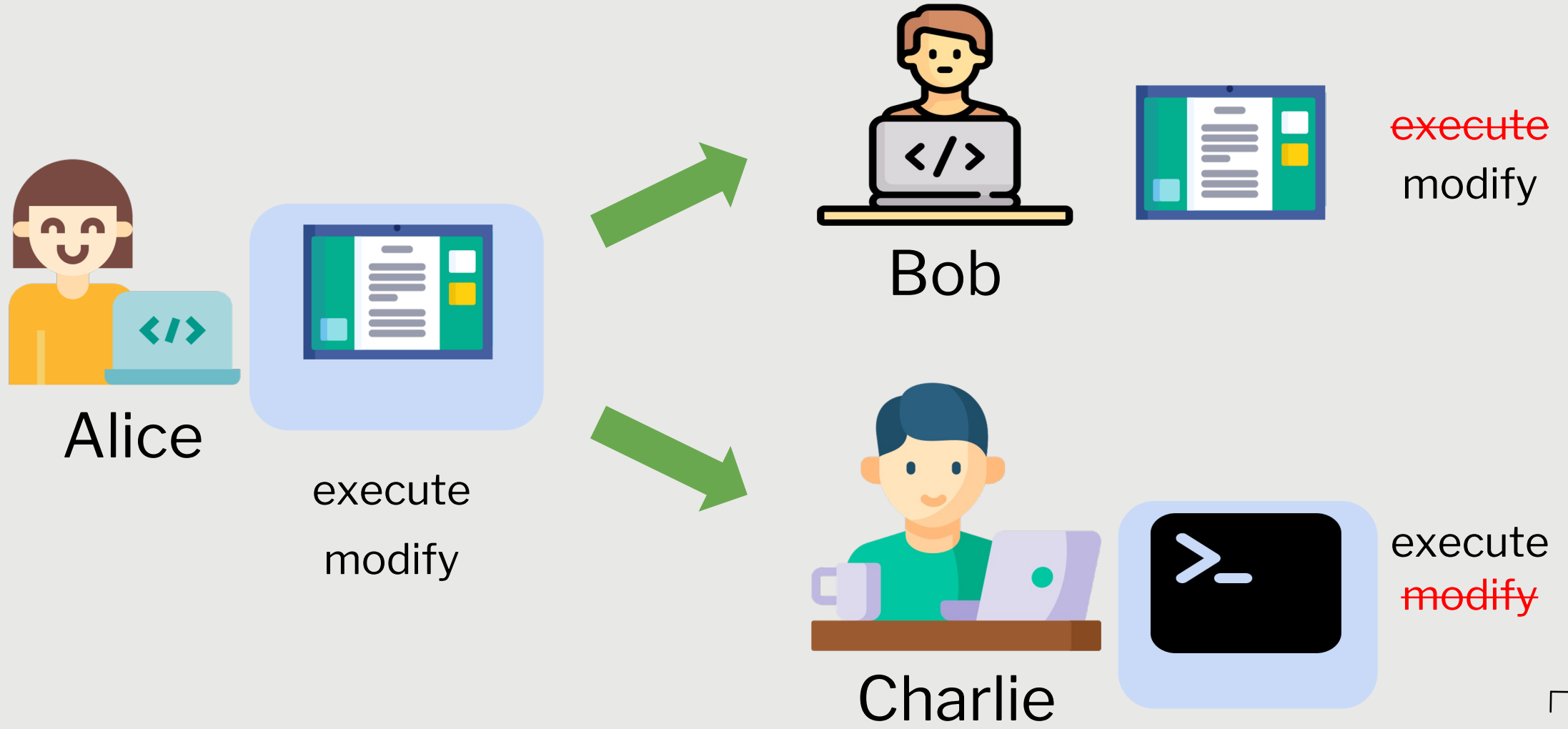


Efficient and
Easy to Use



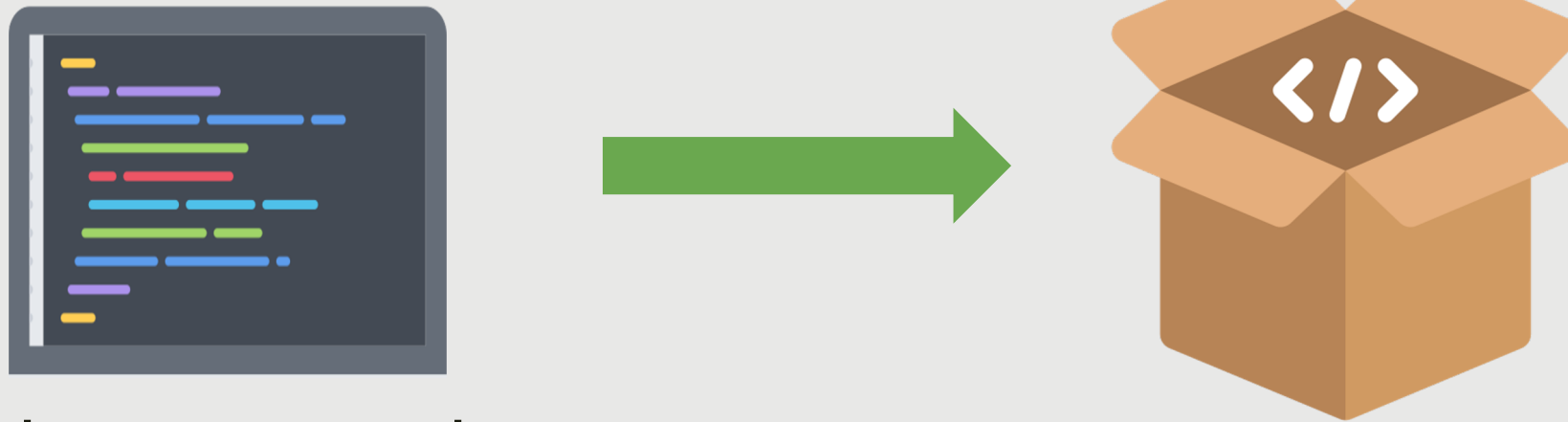


Notebook Reproducibility



Solution: Application Virtualization

- Application Virtualization(AV) is a method to convert executing programs into self-contained packages.



- Works in two modes:
 - Audit Mode - capture and encapsulate execution
 - Repeat Mode - repeat the captured execution

Limitations of AV

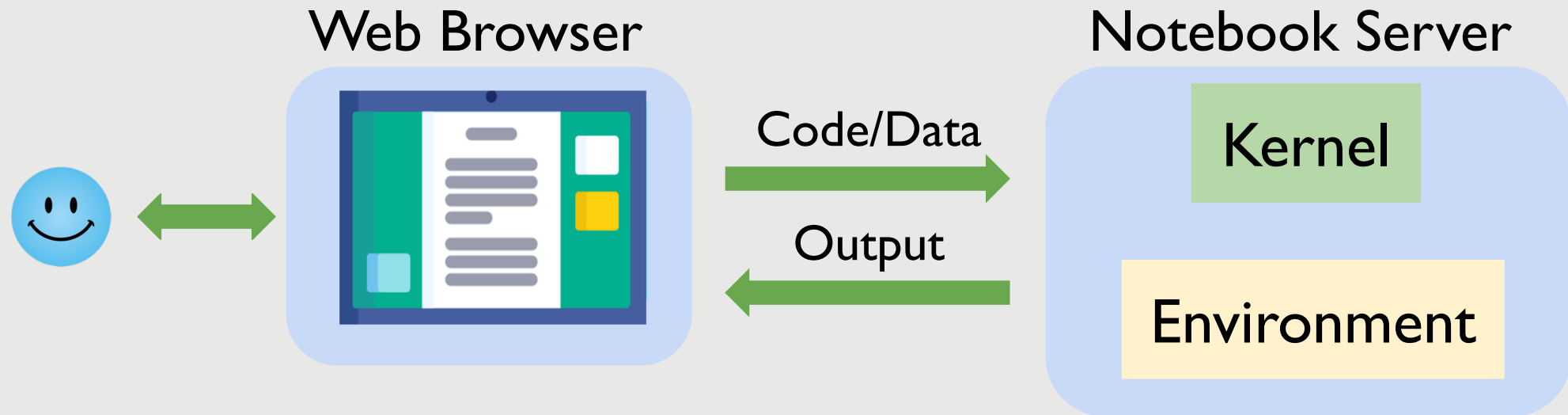
- Most AV systems assume batch workflows.
 - Not suited to capturing interactive notebooks.
- Notebooks need to be converted to batch scripts





Notebook Client Server Architecture

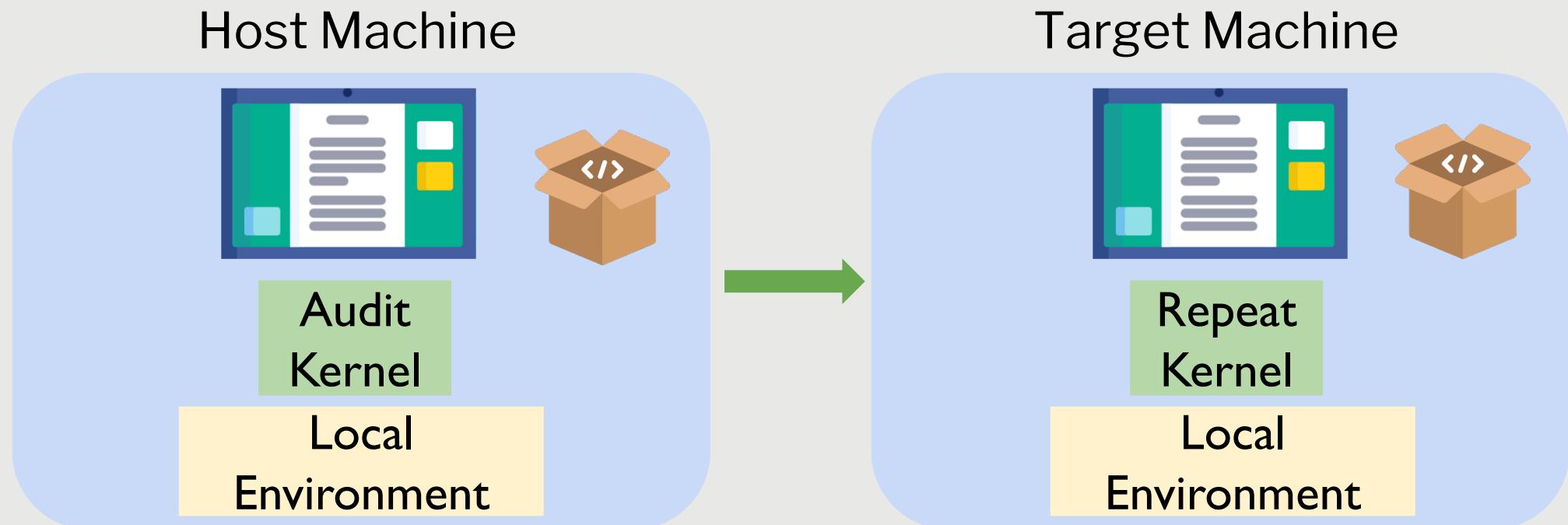
- Extends application virtualization(AV) to the client-server architecture of notebooks.





Extend AV to Notebook Client-Server Architecture

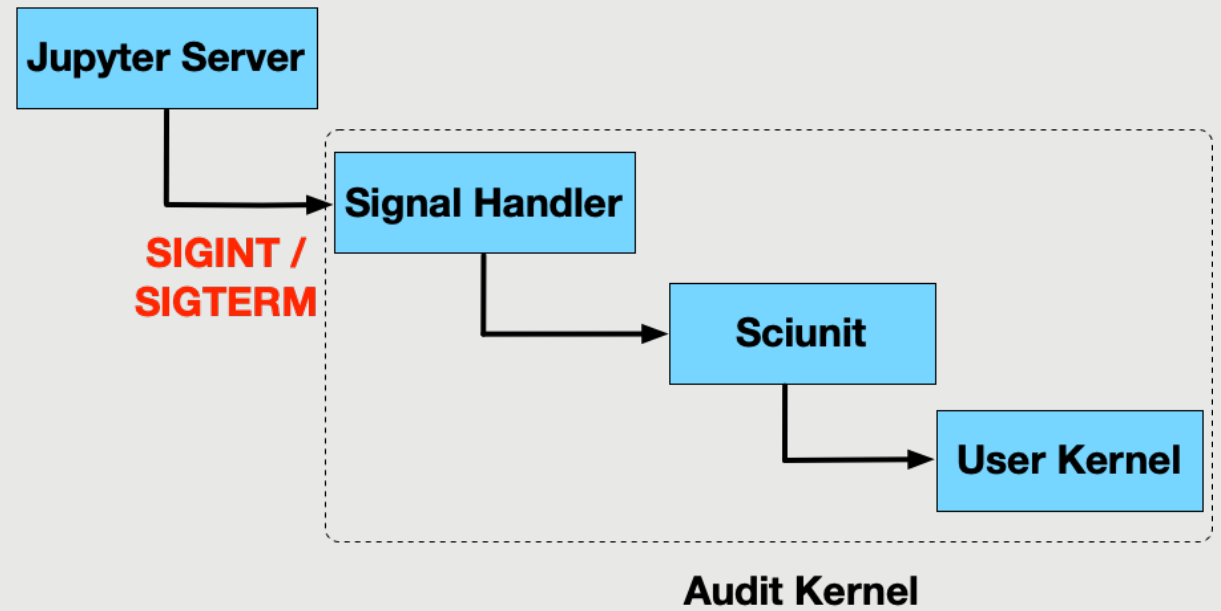
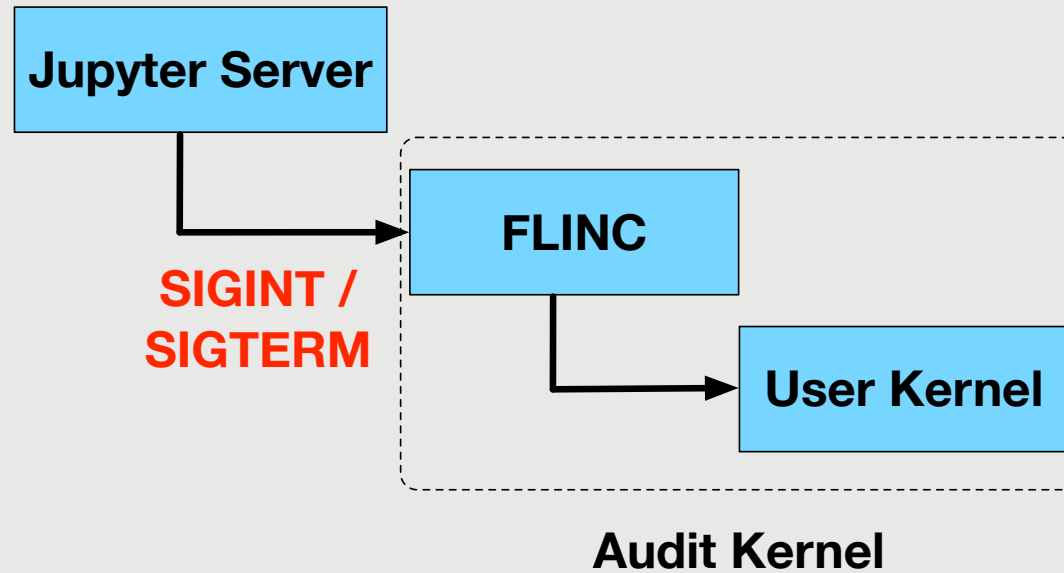
- Creates two additional kernels:
 - *Audit kernel* - executes and audits notebook code
 - *Repeat kernel* – repeats the containerized code





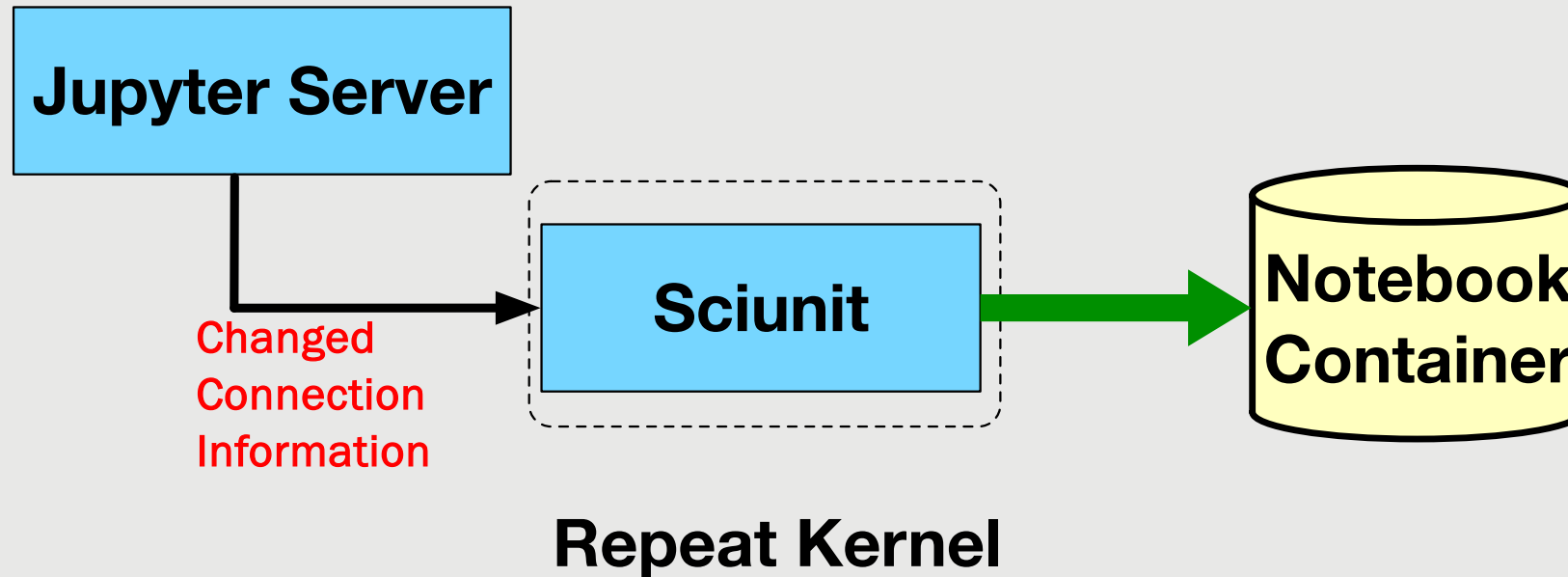
FLINC

Audit Kernel





Repeat Kernel





But, is Extending AV Enough? NO!



Bob

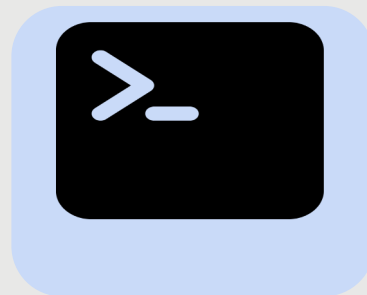


execute 
modify

1. No Guarantee of Reproducible Execution
 - Notebook and container are separately shared

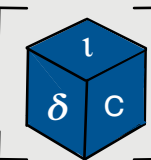


Charlie



execute 
modify

2. No Flexibility with Reproducible Execution
 - a. No way to export dependencies





Ensuring Reproducible Execution

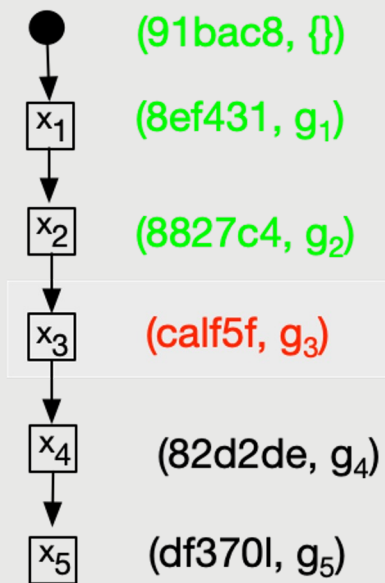
- **Key Idea: Lineage-based notebook cell equivalence**
- Compare *per cell* lineage during audit time with repeat time
 - Cell lineage = cell code hashes and state lineage
 - State lineage =
 - the predecessor cell's lineage
 - the sequence of system events triggered by program instructions in the cell, and
 - the hashes of the associated external data dependencies.



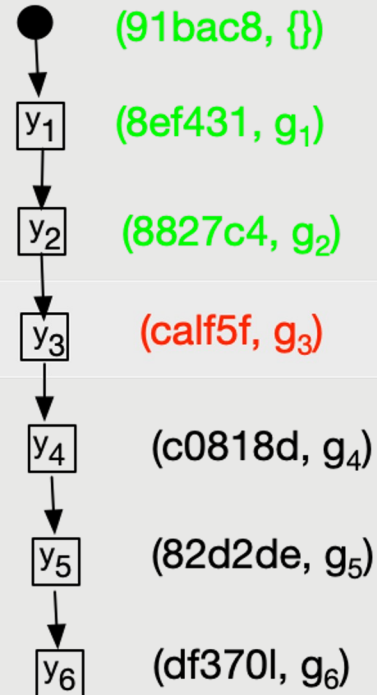


Lineage Equivalence

Lineage of notebook
execution at time t_1



Lineage of notebook
execution at time t_2



Given two notebook executions E_1 and E_2 , cell state ps_i in E_1 is equal to cell state ps_j in E_2 , denoted $ps_i = ps_j$, iff

(i) $h_i = h_j$, and

(ii) $g_i = g_j$

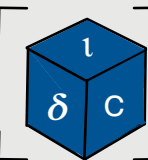
```
4183 fork 4184
4184 exec '/usr/torch/dataloader.py'
4184 open '/home/alice/new_fashion'
4183 mem 567894567
4184 read b2e1772
```

E_3 in L_1

```
6254 fork 6255
6255 exec '/usr/torch/dataloader.py'
6255 open '/home/alice/new_fashion'
6255 read 6789b34
6254 mem 589715363
```

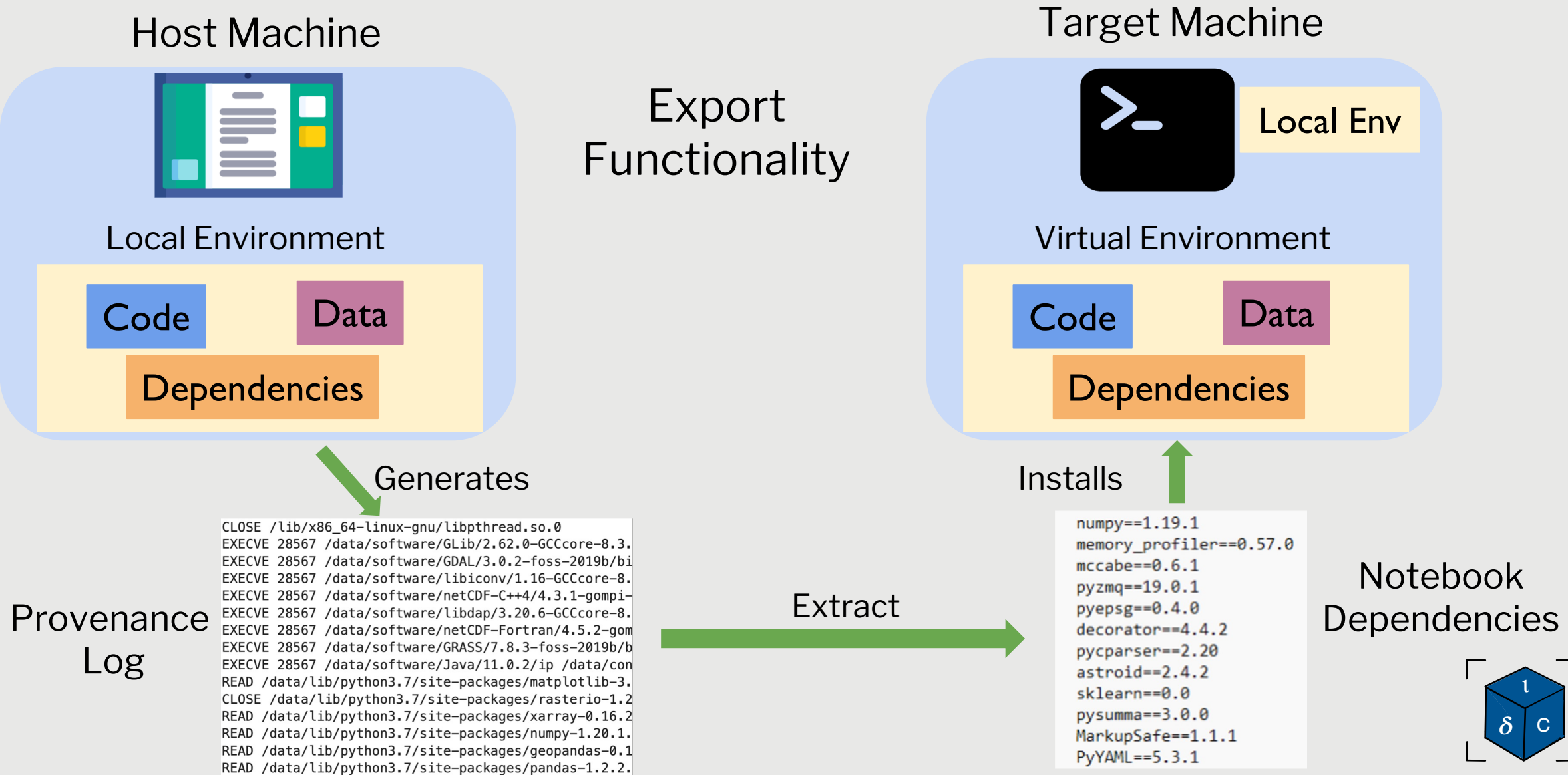
E_4 in L_2

Using Provenance to Improve Computational Reproducibility, ReWords, eScience, 2023





Flexible Reproducibility





Experiments

- Analyze storage and running time for AV and export.
- Experiment using a large dataset of Python notebooks.
- Compare three methods for AV performance.
 1. *Interactive-AV: FLINC notebook auditing*
 2. *Batch-AV: Python file of notebook audited using AV*
 3. **Baseline:** *No-AV: Regular notebook execution*





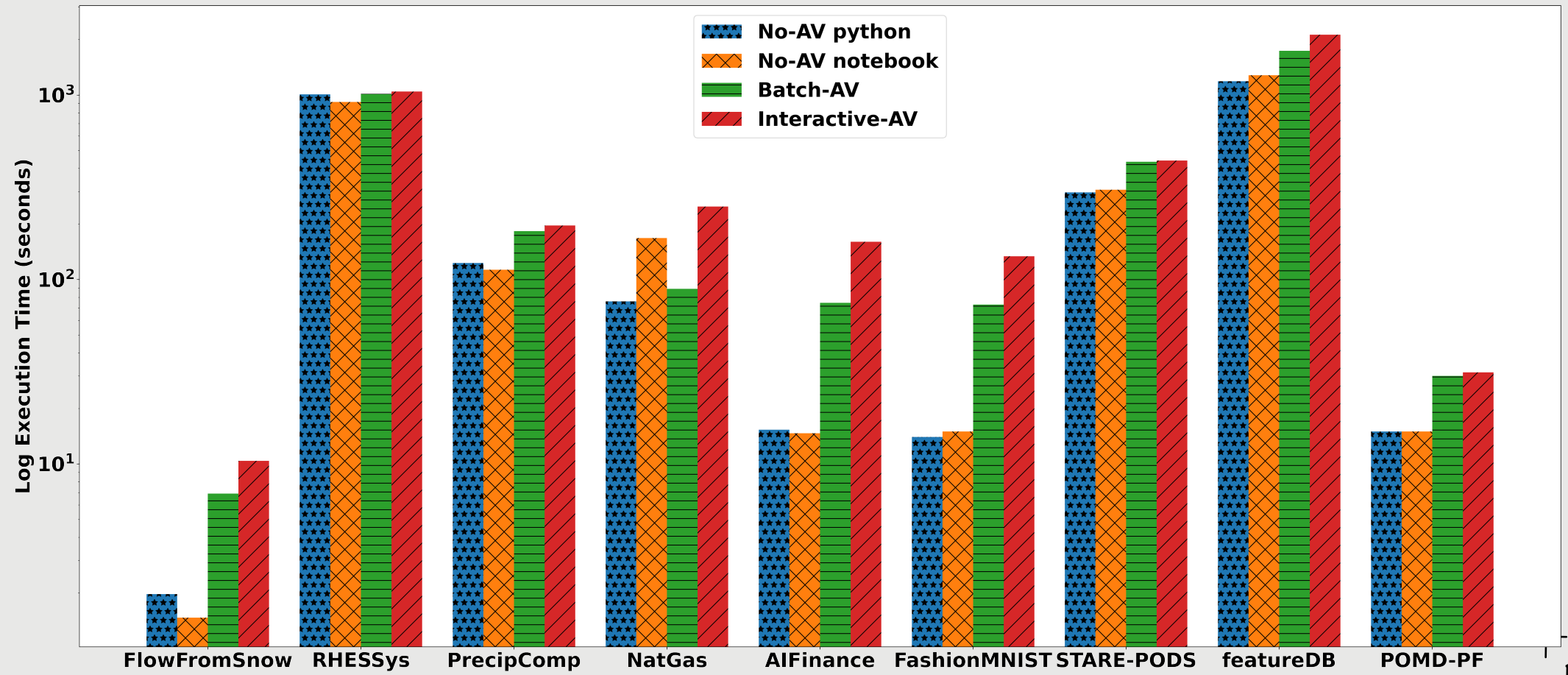
Notebook Dataset

Notebook	Discipline	Size w/ Output	Size w/o Output	Primary Dependencies	Description
FlowFromSnow	Hydrology	105K	6.5K	NumPy, Matplotlib	Simple linear regression analysis on USGS streamflow data and climate data.
HAND	Hydrology	515K	6.9K	NumPy, Matplotlib, TauDEM, RasterIO, GeoPandas	Calculates height above the nearest drainage (HAND) using digital elevation model.
RHESSys	Hydrology	133.1K	15.4K	RHESSys, NetCDF, Pandas	Executes the RHESSys model to run a simulation of the Sagehen creek watershed and conduct sensitivity analysis.
PrecipComp	Hydrology	408.6K	26.6K	Cartopy, GeoPandas, Rioxarray, Shapely	Compares precipitation estimates for the Logan River watershed using two datasets AORC v1.1 and PRISM.
NatGas	Data Science	1263.6K	39.9K	NumPy, Matplotlib, openpyxl	Market analysis of natural gas by running market simulation of 35 years based on economic indicators.
AIFinance	Data Science	35.8K	9.2K	NumPy, Matplotlib, Scikit-learn, Keras, TensorFlow	Deep neural network to predict stock prices in the short term using financial news data.
FashionMNIST	Data Science	13.3K	7.2K	NumPy, Matplotlib, Torch Vision	Image classification using convolutional neural networks on the Fashion-MNIST dataset.
STARE-PODS	Earth Science	419.8K	30.7K	PySARE, STAREPandas, PySTAREPlotlib, GeoPandas, Cartopy	Reads US states geographical data and converts them into STARE trixels, chooses one event, and performs various calculations.
featureDB	Earth Science	9.2K	6.1K	CC3D, NETCDF4, NumPy, Pandas	Loads precipitation data from IMERG satellite and runs a connected components algorithm to find and label connected areas.
POMD-PF	Earth Science	25.6K	25.6K	NumPy, Pandas, PyHDF, SciPy, Cartopy	Compares data from IMERG satellite with simulated GEOS5 data by subsetting most significant rain values.



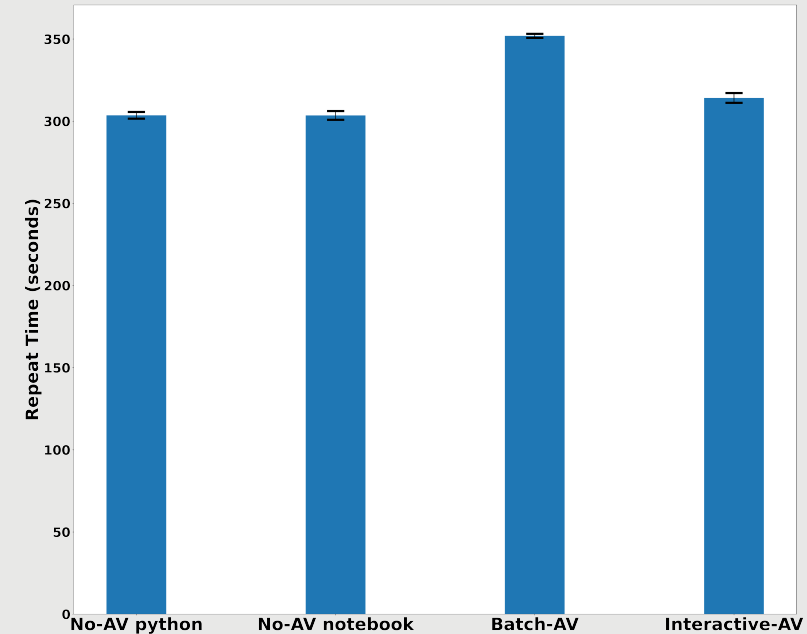
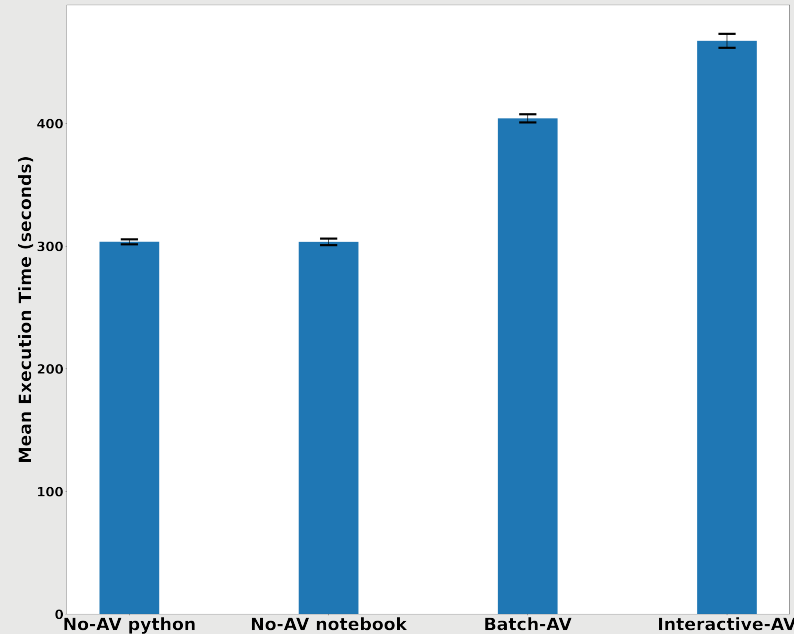


Notebook Execution Time





Average Execution Time

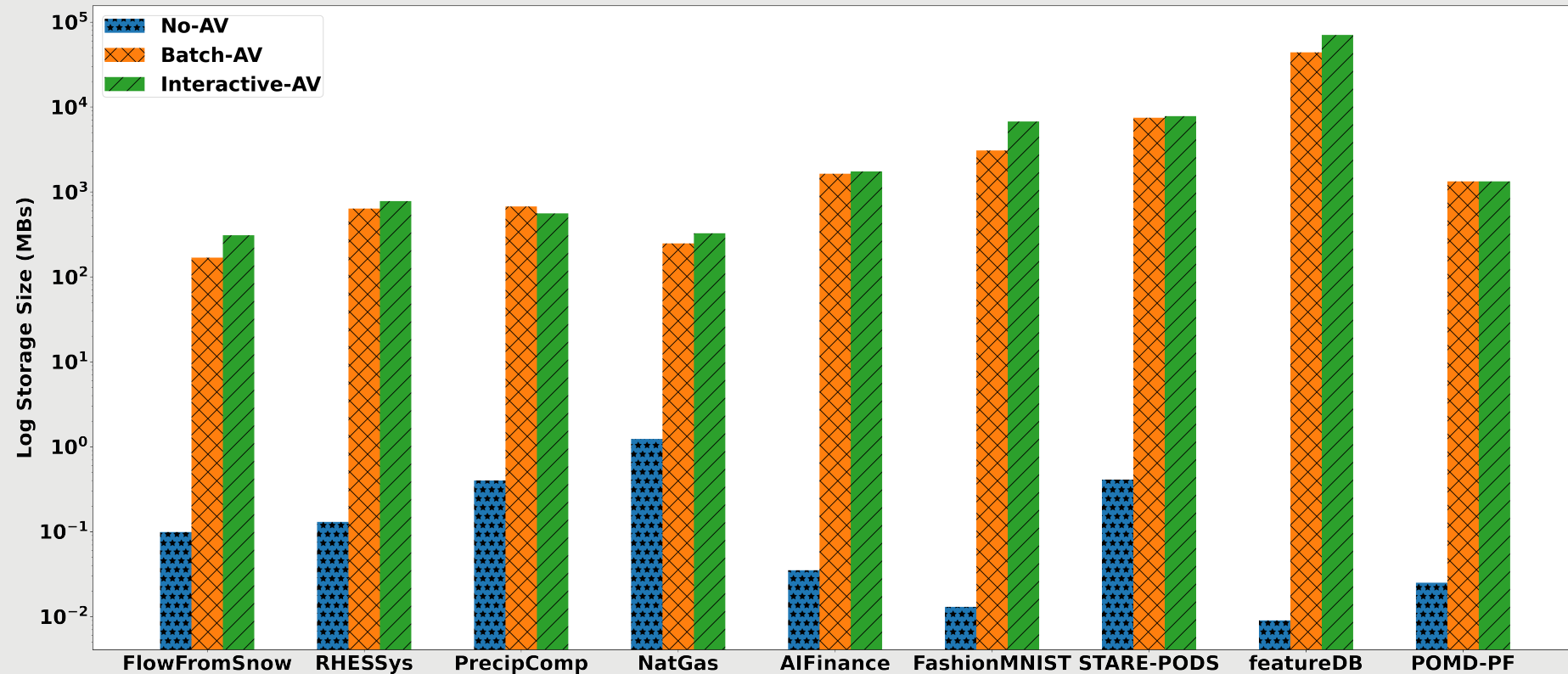


- Audit: About 16% overhead of interactive AV than batch AV.
- Repeat: Interactive-AV method adds an overhead of only around 3.5% compared to the actual notebook execution, but batch-AV is more expensive.
 - *Almost 11% less than the time it takes to repeat the same notebook using batch-AV method.*

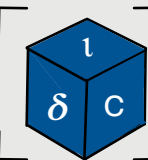




Storage size of notebook containers

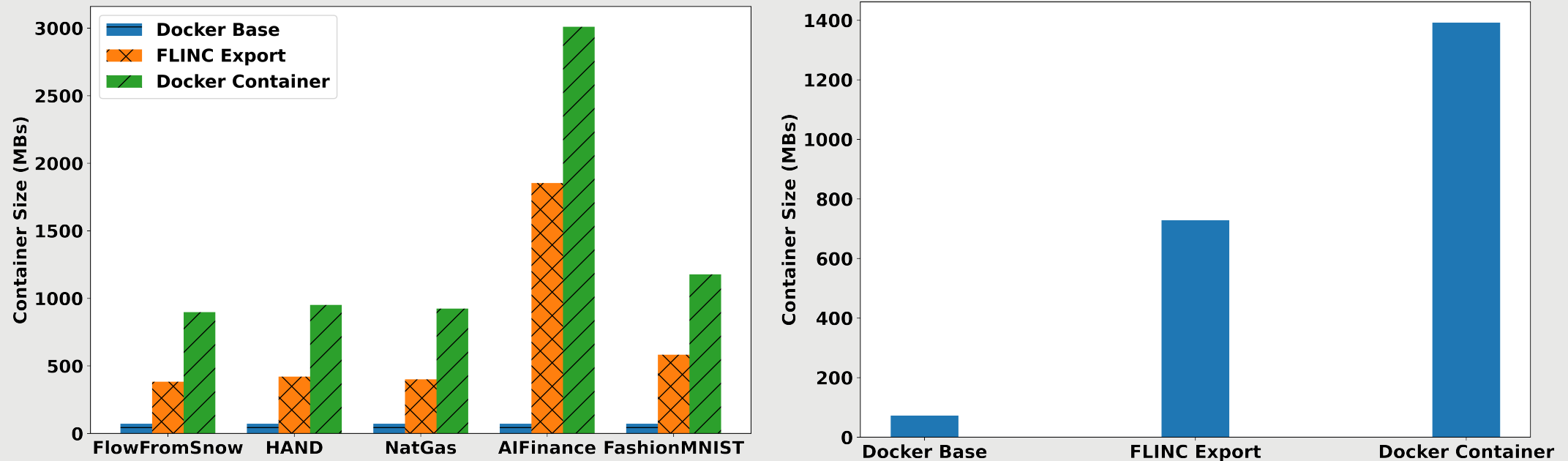


- 34% larger in size as compared to the containers created using Batch-AV.
- Intermediate data cached by notebook execution accounts for the primary increase in size.

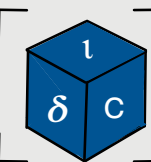




Notebook Containers Vs Docker

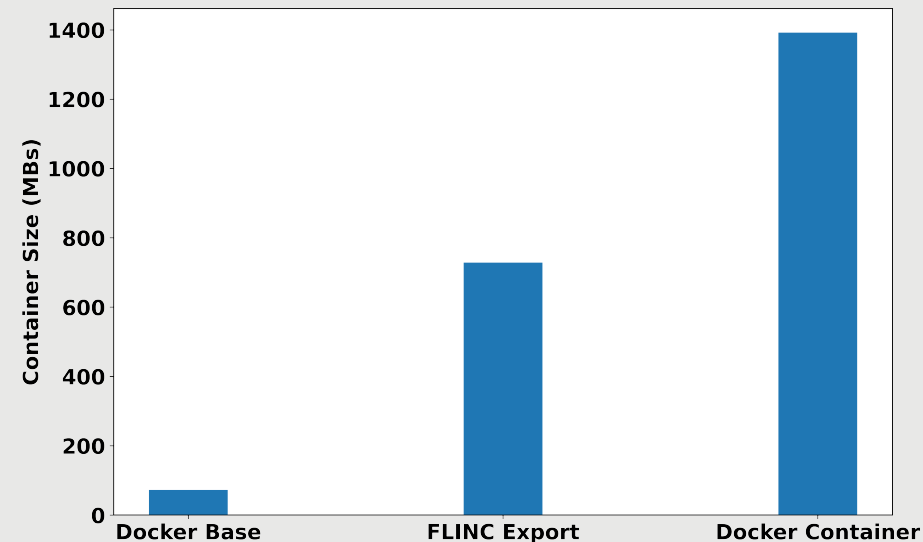
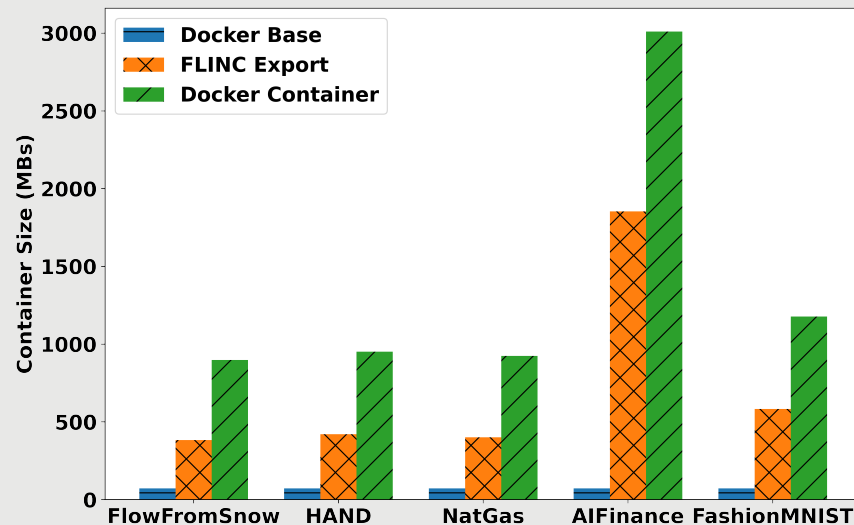


46% less time than Docker to create a new virtual environment on the target system and install the required dependencies.

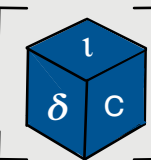




Notebook Containers Vs Docker (2)



- Docker containers are, on average, 91% larger in size for all our notebooks compared to the containerized environment created by FLINC.





Summary

- Establishing reproducibility of scientific claims is critical.
 - Process for evaluation of scientific claims is being rethought.
 - Artifact description and evaluation are becoming part of conferences
 - But more efficient tools are needed.
- Containers will be a prominent choice, but their reliability is poor
 - Dependencies must be specified
 - Inefficient to use
 - No guarantees for execution verification
 - Not meant for interactive programs
- Use of provenance as critical for repeating and running multiple executions.
 - *System call based provenance helps to establish execution equivalence when inputs are controlled.*
 - *Content hashing is a vital component to establish this equivalence*
 - *Both process and file versioning is required when inputs change.*





2023 ACM Conference on Reproducibility and Replicability

June 27-29, 2023

UC Santa Cruz | Santa Cruz, California | #acmrep

This is a *hybrid event* – remote participation is welcome.

[Keynote Speakers ->](#)

Contact: acmrep23-info-group@ucsc.edu

Conference Organizers

General Chairs

Carlos Maltzahn (UC Santa Cruz)

Philippe Bonnet (IT University of Copenhagen)

Program Chairs

Tanu Malik (DePaul University)

Jay Lofstead (Sandia National Laboratories)

Tutorial Chair

Tweets from @acmrep



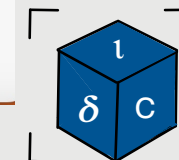
ACM REP '23
@acmrep · Apr 1



We are very excited to have [@thoefler](#) confirmed as keynote speaker at [#acmrep23](#). The title of his keynote will be "Reproducing Performance - The Good, the Bad, and the Ugly."

CM F

acm-rep.git...
Featured
Keynote ...





Roadmap: A Formal Lineage-based Framework for Reproducible Research

Extension to different types of computations

- Parallel computations
- Interactive computations
- Distributed computations

Making package complete for reproducible research

- Automatic dependency documentation
- Automatic generation of citations

Improving package sizes

- Improving container sizes
- Container-level provenance

Infrastructure integrations and usability

- Integration with Hydroshare
- Increasing usability





Acknowledgements

- NSF CNS-1846418, ICER-1639759, ICER-1661918.
- DOE BSSW Fellowship
- NASA AIST Award 210000095





Acknowledgements



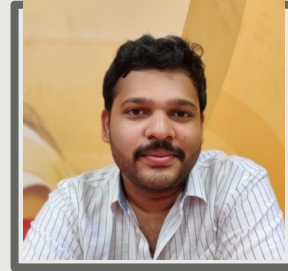
Yuta Nakamura
Ph.D. student



Raza Ahmad
Ph. D Student



Jason Chuah
M.S. student



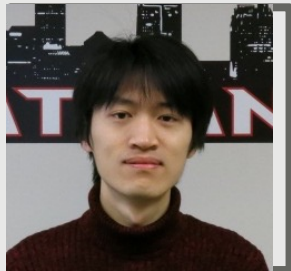
Nithin Manne
M.S Student



Rohan Tikmany
M.S Student



Ton That Dai Hai
Postdoctoral Associate



Zhihao Yuan
Research Engineer



Ian Foster
UChicago & ANL



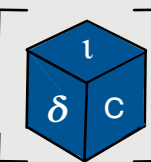
Ashish Gehani
SRI International



Iyad Kanj
DePaul



Amitabha Bagchi
IITD





Thank You

- Please check out our work at
 - <https://dice.cs.depaul.edu/>
 - <https://github.com/depaul-dice>
- Questions/Contact: tanu.malik@depaul.edu

